

Java OOP Mastery - Phase 5

THREADING & MULTITHREADING | Runnable | Thread | start() vs run() | Thread names | sleep() | join() | Race Conditions | synchronized | volatile | ExecutorService

Goal: Understand what a thread is, how Java starts it, how Runnable and Thread relate, why output ordering is unpredictable, and why synchronization is necessary.

1. Process vs Thread

A Java program runs as a process, and inside that process there are one or more threads. The most familiar one is the main thread.

```
Process
|
+-- main thread
+-- worker thread
+-- worker thread
```

A **thread** is an independent path of execution inside a process. **Multithreading** means a program has multiple threads that can make progress concurrently.

2. Why Use Multiple Threads?

```
Single thread:
download -> finish -> send email -> finish -> report
```

```
Multiple threads:
Thread A -> download
Thread B -> send email
Thread C -> generate report
```

The goal is to allow independent work to overlap. This is especially useful for I/O-heavy tasks and for using multiple CPU cores when parallel execution is possible.

3. Runnable - The Work / Task

Runnable is a Java interface representing a unit of work. Its core method is run(). A class can implement Runnable without giving up its one superclass inheritance slot.

```
class Task implements Runnable {

    @Override
    public void run() {
        System.out.println("Task running");
    }
}
```

Mental model:

```
Runnable
|
+-- WHAT work should be done?
```

4. Thread - The Execution Mechanism

A Thread object represents a thread of execution. You can give it a Runnable task to run.

```
Runnable task = new Task();

Thread t1 = new Thread(task);

t1.start();
```

This separates the **job** from the **worker executing the job**.

5. The Important Runnable + Thread Example

```
class Task implements Runnable {  
  
    @Override  
    public void run() {  
  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(  
                Thread.currentThread().getName()  
                + " : " + i  
            );  
        }  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Thread t1 = new Thread(new Task(), "Thread-A");  
        Thread t2 = new Thread(new Task(), "Thread-B");  
  
        t1.start();  
        t2.start();  
    }  
}
```

Possible output:

```
Thread-A : 1  
Thread-B : 1  
Thread-B : 2  
Thread-A : 2  
Thread-A : 3  
Thread-B : 3  
...
```

The exact order is **not guaranteed**. The scheduler may interleave the threads differently from run to run.

6. Why Can We Pass "Thread-A" If Task Has No Name Constructor?

Because the string is **not being passed to Task**. It is being passed to Java's built-in Thread constructor.

```
Thread t1 = new Thread(new Task(), "Thread-A");
```

There are two constructions happening conceptually:

```
new Task()  
-> creates your Runnable task object  
  
new Thread(task, "Thread-A")  
-> creates Java's Thread object  
-> gives that Thread object the name "Thread-A"
```

Your Task class does not need a constructor that accepts a String. The built-in Thread class stores/manages the thread name.

Important: Thread.currentThread().getName() asks for the name of the thread that is currently executing that code. It is not asking the Task object for its name.

7. Two Equivalent Ways to Pass the Task

Step-by-step form:

```
Runnable task = new Task();  
Thread t1 = new Thread(task, "Thread-A");
```

Inline form:

```
Thread t1 = new Thread(new Task(), "Thread-A");
```

Both are valid. The first gives the Runnable object a local reference named task; the second creates the Task object and passes the reference directly into the Thread constructor. The inline object is often described informally as an unnamed/anonymous object because no variable stores its reference.

8. start() vs run() - NEVER Mix These Up

```
t1.start();
```

start() starts a new thread of execution. The new thread then invokes the task's run() method.

```
t1.run();
```

Calling run() directly is just a normal method call on the current thread. It does **not** create a new thread.

```
start()
-> new thread begins
-> run() executes on that thread

run()
-> ordinary method call
-> current thread executes it
```

Interview trap: Never say that run() creates a thread. start() is the operation that starts a new thread.

9. Why Is Output Mixed?

Once multiple threads are runnable, the scheduler determines how they are scheduled. A thread may run for some period, pause, and another thread may run. You must not assume a fixed ordering unless your program explicitly establishes one.

```
Thread-A : 1
Thread-B : 1
Thread-A : 2
Thread-B : 2
...
```

This is concurrent execution. Whether two threads are literally executing at the same instant depends on available CPU cores and scheduling; concurrency and parallelism are related but not identical concepts.

10. sleep()

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

Thread.sleep(1000) pauses the **current thread** for approximately 1 second (subject to scheduling). It does not pause every thread in the JVM.

11. join()

```
Thread t1 = new Thread(task);

t1.start();
t1.join();

System.out.println("Main finished");
```

join() makes the calling thread wait for the target thread to terminate before continuing. In this example, the main thread waits for t1.

12. Shared State - Where Things Get Dangerous

```
class Counter {
    int count = 0;

    void increment() {
        count++;
    }
}
```

The expression `count++` is not a single indivisible operation. Conceptually it involves reading the old value, adding one, and writing the new value.

```
read count
->
add 1
->
write count
```

13. Race Condition

If two threads operate on the same shared variable at the same time, their operations can interleave in an unsafe way.

```
count = 0

Thread A: read 0
Thread B: read 0

Thread A: write 1
Thread B: write 1

final count = 1
```

Two increments happened, but one update was lost. This timing-dependent bug is a **race condition**.

14. synchronized

```
class Counter {
    private int count = 0;

    synchronized void increment() {
        count++;
    }

    int getCount() {
        return count;
    }
}
```

A `synchronized` instance method uses the object's monitor as a lock. For calls targeting the same object, one thread at a time can execute the synchronized critical section.

```
Thread A -> lock -> increment -> unlock
Thread B -> waits -> lock -> increment -> unlock
```

Important: Synchronization should protect a logical critical section consistently. Locking one access path while leaving another unsynchronized can still leave a race.

15. volatile - Visibility, Not General Thread Safety

```
volatile boolean running = true;
```

`volatile` is primarily about visibility: reads and writes to the volatile variable have the required cross-thread visibility semantics. It does **not** make a compound operation like `count++` automatically atomic.

```
volatile
-> visibility
```

synchronized
-> mutual exclusion + visibility guarantees

16. Concurrency vs Parallelism

Concurrency: multiple tasks make progress during overlapping time periods. **Parallelism:** multiple tasks actually execute simultaneously, typically on different CPU cores.

Concurrency -> about dealing with multiple tasks
Parallelism -> about simultaneous execution

17. Thread Lifecycle - Useful Mental Model

```
NEW
|
v
RUNNABLE
|
v
executing / scheduled
|
+--> BLOCKED
+--> WAITING
+--> TIMED_WAITING
|
v
TERMINATED
```

The exact JVM state terminology matters in the Java API. A thread becomes eligible to run after start(); it does not mean your program can directly force the OS to run it at a precise instant.

18. ExecutorService - Real-World Direction

```
ExecutorService executor =
    Executors.newFixedThreadPool(3);

executor.submit(() -> {
    System.out.println("Task 1");
});

executor.submit(() -> {
    System.out.println("Task 2");
});

executor.shutdown();
```

Instead of creating a new raw Thread for every small task, applications commonly use executors and thread pools. The pool manages reusable worker threads while you submit tasks.

Mental model:

```
Task
|
v
ExecutorService
|
v
thread pool
|
+-- worker thread
+-- worker thread
+-- worker thread
```

19. Quick Rules to Permanently Remember

Runnable
-> represents work / task

Thread

-> execution path / worker

`new Thread(task, "Thread-A")`
-> Thread constructor receives the Runnable and name

`start()`
-> starts a new thread

`run()`
-> task body; direct call does NOT create a new thread

`currentThread().getName()`
-> name of the thread currently executing the code

`sleep()`
-> pause current thread

`join()`
-> wait for another thread to finish

race condition
-> unsafe result caused by concurrent access/interleaving

synchronized
-> protect a critical section with a monitor lock

volatile
-> visibility; NOT a replacement for atomic synchronization

ExecutorService
-> manage pools of worker threads and submitted tasks

20. Phase 5 Challenge + Interview Check

- Q1.** Explain exactly what happens when `t.start()` is called compared with `t.run()`.
- Q2.** Why can two threads both executing `count++` lose an update?
- Q3.** Explain why `Runnable task = new Task();` is valid and why it is often preferred to extending `Thread` for the task itself.
- Q4.** In `new Thread(new Task(), "Thread-A")`, which object receives the string `"Thread-A"`?
- Q5.** Why can output from two independent threads appear in a mixed order?
- Q6.** What problem does `synchronized` solve that `volatile` by itself does not?
- Q7.** Why does `Thread.currentThread().getName()` show different names when the same `Runnable` is executed by different `Thread` objects?
- Q8.** Explain the difference between concurrency and parallelism in your own words.
- Q9.** When would you prefer `ExecutorService` over manually creating many `Thread` objects?
- Q10.** Predict whether this creates one thread or two new threads: `t1.start(); t2.start();`.

Final mental model:

`Runnable` -> WHAT work?
`Thread` -> WHO executes it?
`start()` -> START a new execution path
`run()` -> DO the work
shared state -> race-condition risk
`synchronized` -> coordinate access
`executor` -> manage many tasks/worker threads